

Deep Residual Learning for Image Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers---8x deeper than VGG nets but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task.

1 Introduction

Deep convolutional neural networks have led to a series of breakthroughs for image classification. Deep networks naturally integrate low/mid/high-level features and classifiers in an end-to-end multi-layer fashion, and the 'levels' of features can be enriched by the number of stacked layers. Recent evidence reveals that network depth is of crucial importance, and the leading results on the challenging ImageNet dataset all exploit 'very deep' models with a depth of sixteen to thirty.

1.1 Degradation Problem

As deeper networks are able to converge, a degradation problem has been exposed: with the network depth increasing, accuracy gets saturated and then degrades rapidly. Such degradation is not caused by overfitting, and adding more layers to a suitably deep model leads to higher training error. We explicitly address this problem by introducing deep residual learning framework.

2 Deep Residual Learning

2.1 Residual Learning

Let us consider $H(x)$ as an underlying mapping to be fit by a few stacked layers, where x denotes the inputs to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate complicated functions, then it is equivalent to hypothesize that they can asymptotically approximate the residual functions. Rather than hope stacked layers approximate $H(x)$, we explicitly let these layers approximate a residual function $F(x) = H(x) - x$. The original function thus becomes $F(x) + x$.

2.2 Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is defined as $y = F(x, \{W_i\}) + x$. Here x and y are the input and output vectors of the layers considered. The function $F(x, \{W_i\})$ represents the residual mapping to be learned. The dimensions of x and F must be equal. If this is not the case, we can perform a linear projection Ws by the shortcut connections to match the dimensions: $y = F(x, \{W_i\}) + Ws$.

2.3 Network Architecture

We use a batch normalization (BN) right after each convolution and before activation. We adopt the ResNet-50 architecture with bottleneck blocks. Each residual function F has three layers: 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then increasing (restoring) dimensions.